# A GPT-2 Transformer Written From Scratch

Implementation, Theory and Experiments

Hizkeal Yeshitila

November 11, 2026

**Writing standard.** I write this report in Simplified Technical English (ASD-STE100). Sentences are short. I use the active voice. I use one term for one meaning.

## Summary

I implemented a decoder-only Transformer in PyTorch. I wrote the attention, the layer normalization, the feed-forward network and the blocks from basic tensor operations. I did not use a pretrained model. I did not use a library Transformer block.

I trained the model on SMILES strings. A SMILES string is a molecule written as text. Aspirin is `CC(=O)OC1=CC=CC=C1C(=O)O`. The task is next-character prediction.

This dataset has one useful property. The model must learn a strict grammar to score well. Brackets must close. Ring-opening digits must pair. Every atom must keep a legal valence. RDKit parses the output, or it does not. My result is therefore a number and not an opinion.

## Contents

**1 Understanding *Attention Is All You Need*** **3**
- 1.1 The problem that the Transformer solves . . . 3
- 1.2 The difference from RNNs and LSTMs . . . 3
- 1.3 Scaled dot-product attention . . . 3
- 1.4 The reason for multiple heads . . . 4
- 1.5 Positional encodings . . . 4
- 1.6 Feed-forward networks, layer normalization and residual connections . . . 4

**2 From the Original Transformer to GPT-2** **5**
- 2.1 The encoder-decoder architecture . . . 5
- 2.2 The reason for a decoder-only architecture in GPT-2 . . . 5
- 2.3 The three types of attention . . . 5
- 2.4 How causal masking makes autoregressive generation possible . . . 5
- 2.5 Architectural differences . . . 6

**3 Implementation Explanation** **6**
- 3.1 The format of one training sequence . . . 6
- 3.2 Notation and model size . . . 7
- 3.3 The model architecture . . . 7
- 3.4 Inside causal self-attention . . . 7
- 3.5 The loss . . . 9
- 3.6 Text generation . . . 9

**4 Experiments and Findings** **9**
4.1 Training and validation loss . . . . . 9
4.2 The effect of temperature . . . . . 10
4.3 The effect of prompt length and context . . . . . 10
4.4 The effect of the architecture . . . . . 12
4.5 Limitations . . . . . 12
4.6 Implementation challenges . . . . . 13
4.7 Lessons learned . . . . . 14

**References** **14**

# 1 Understanding *Attention Is All You Need*

## 1.1 The problem that the Transformer solves

Earlier sequence models read one token at a time. This causes two problems.

The first problem is speed. A recurrent model computes position $t$ only after position $t-1$. The work inside one training example cannot run in parallel. A long sequence is therefore slow.

The second problem is distance. Information from token 1 reaches token 100 through 99 steps. The gradient must travel the same 99 steps backward. The signal becomes weak over this distance.

The Transformer removes both problems. Attention connects every position to every other position directly. The path length between any two tokens is constant. All positions compute together.

## 1.2 The difference from RNNs and LSTMs

| **Property** | **RNN / LSTM** | **Transformer** |
|---|---|---|
| Computation order | one position after another | all positions together |
| Sequential steps per example | $O(n)$ | $O(1)$ |
| Path between two tokens | up to $n$ steps | 1 step |
| Memory of the past | one hidden state vector | direct access to all positions |
| Cost per layer | linear in $n$ | quadratic in $n$ |

Table 1: The main differences between a recurrent model and a Transformer.

An LSTM adds gates to the recurrent cell. The gates let the model keep information for longer. This reduces the distance problem, but it does not remove it. The path is still sequential.

The Transformer pays a price for this. Attention compares every position with every other position. The cost is therefore quadratic in the sequence length. My model limits the context to 128 tokens for this reason.

## 1.3 Scaled dot-product attention

Attention gives each position a mixture of information from other positions. The model learns which positions to use.

Each token produces three vectors:

- The **query** shows what the token must find.
- The **key** shows what the token can offer.
- The **value** holds the content that the token sends.

The dot product of a query and a key gives a similarity score. A high score means that the two tokens are relevant to each other. The softmax function turns the scores into weights. The weights are positive, and they sum to 1. The output is the weighted sum of the values.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{1}$$

The division by $\sqrt{d_k}$ is necessary. The dot product adds $d_k$ products together. The variance of the result grows with $d_k$. Large scores push the softmax function into a flat region. In that region the gradient is almost zero, and the model learns slowly. The division keeps the variance near 1.

I implement Equation 1 in `gpt/model.py:52-65`. My head dimension is 32, so I divide by $\sqrt{32}$.

### 1.4 The reason for multiple heads

One attention operation gives one set of weights. The output is one average over the positions. Different relations must then share the same average, and the average hides them.

Multi-head attention splits the model width into $h$ parts. Each head has a dimension $d_k = d_{\text{model}}/h$. Each head computes its own attention in its own subspace. The model concatenates the outputs and projects them once.

The total cost stays almost the same. The model gains the ability to track several relations at the same time. In my model, one head can follow a ring-opening digit. Another head can follow an open bracket.

My `n_head` experiment tests this directly. I hold `n_embd` at 256 and I change `n_head`. More heads therefore means a smaller dimension in each head. Section 4.4 gives the result.

### 1.5 Positional encodings

Attention has no sense of order. It computes a weighted sum over a set of positions. If you change the order of the input tokens, the set of outputs does not change. The attention operation is permutation-equivariant.

Order is essential for my task. The string `C1CCCCC1` is a ring. The first digit `1` opens the ring, and the second digit `1` closes it. A model without order information cannot tell which digit comes first. It cannot learn the grammar.

A positional encoding adds order information to each token. The original paper uses fixed sinusoidal functions of the position. The functions have different frequencies. The model can therefore infer relative distance from the values.

I use learned positional embeddings, as GPT-2 does. My model holds a table of 128 vectors. Vector $t$ holds the information for position $t$. The model learns the table with the other weights. Section 2.5 compares the two methods.

### 1.6 Feed-forward networks, layer normalization and residual connections

**The feed-forward network.** Attention mixes information across positions. However, the output is a weighted sum of the values, so attention is linear in the values. The model needs a non-linear step. The feed-forward network supplies it. The network expands each position from 256 to 1024 dimensions, applies GELU, and projects back to 256. It acts on each position separately. I implement it in `gpt/model.py:67-81`.

**Layer normalization.** The scale of the activations changes during training. Large or small activations make the gradient unstable. Layer normalization fixes the scale. It computes the mean and the variance over the features of one token. It then normalizes that token. It does not use the batch, so it works with any batch size. I implement Equation 2 in `gpt/model.py:13-30`.

$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta \tag{2}$$

**Residual connections.** Each sublayer adds its output to its input. The model computes $x = x + \text{Sublayer}(x)$ and not $x = \text{Sublayer}(x)$. This gives two benefits. The gradient has a direct path to the early layers, so deep models train. Each block also learns a small change to a shared representation. It does not learn a replacement. I implement the two residual connections in `gpt/model.py:94-95`.

# 2 From the Original Transformer to GPT-2

## 2.1 The encoder-decoder architecture

The original paper solves machine translation. A translation task has two different sequences. The architecture therefore has two stacks.

The **encoder** reads the source sentence. Each encoder layer has unmasked self-attention and a feed-forward network. Every source token can see every other source token. The encoder output is a representation of the complete source.

The **decoder** writes the target sentence. Each decoder layer has three sublayers:

1. Masked self-attention over the target tokens that it already produced.
2. Cross-attention to the encoder output.
3. A feed-forward network.

The decoder must not see future target tokens. The mask prevents this.

## 2.2 The reason for a decoder-only architecture in GPT-2

Language modelling has only one sequence. The context is the text before the current token. There is no separate source sentence.

Two parts of the original architecture then have no input. The encoder has no source to read. The cross-attention sublayer has no encoder output to attend to. GPT-2 removes both parts.

Only the masked self-attention decoder remains. My model follows this design. It has token embeddings, 8 identical blocks, a final layer normalization and an output head.

## 2.3 The three types of attention

| Type | Query from | Key and value from | Mask | Used in |
|---|---|---|---|---|
| Masked self-attention | the sequence | the sequence | causal | GPT-2; my model |
| Unmasked self-attention | the sequence | the sequence | none | encoder; BERT |
| Cross-attention | the decoder | the encoder output | none over source | not in my model |

Table 2: The three types of attention differ in two ways only.

The three types use Equation 1. They differ in the source of the keys and the values. They also differ in the mask.

Unmasked self-attention is correct for an encoder. The encoder reads a complete input, so the future is available and useful. Masked self-attention is necessary for generation. A generator has no future to read.

## 2.4 How causal masking makes autoregressive generation possible

The mask is a lower-triangular matrix of ones. I build it with `torch.tril` in `gpt/model.py:49-50`. I apply it in `gpt/model.py:59`.

I set the masked scores to $-\infty$ **before** the softmax. This is important. The exponential of $-\infty$ is 0. The future positions therefore get exactly zero weight. The remaining weights still sum to 1. A mask after the softmax would leave rows that do not sum to 1.

The mask gives position $t$ access to positions 0 to $t$ only. This has a large practical benefit. One forward pass over a sequence of length $T$ produces $T$ predictions. Every prediction uses only legal context. The model therefore trains on all positions at the same time. At generation time the same weights produce one token at a time, from left to right.

Without the mask the model could read the next character from the input. The loss would fall close to zero. Generation would still be impossible, because at generation time the next character does not exist.

### 2.5 Architectural differences

| Aspect | Original Transformer (2017) | GPT-2 and my model |
|---|---|---|
| Structure | encoder and decoder | decoder only (`model.py:99`) |
| Positional information | fixed sinusoidal functions | learned table (`model.py:106`) |
| Normalization placement | post-LN | pre-LN (`model.py:94-95`) |
| Final normalization | none | `ln_f` before the head (`model.py:109`) |
| Activation | ReLU | GELU (`model.py:78`) |
| Output head | a separate matrix | tied to the embedding (`model.py:111`) |
| Residual initialization | standard | scaled by $1/\sqrt{2N}$ (`model.py:115-117`) |

Table 3: The differences between the original Transformer and my implementation.

Two of these differences are important.

**Normalization placement.** The original design normalizes after the residual addition, as $\mathrm{LN}(x + \mathrm{Sublayer}(x))$. The residual path then passes through a normalization in every block. Deep models become difficult to train. GPT-2 normalizes the input of each sublayer instead, as $x + \mathrm{Sublayer}(\mathrm{LN}(x))$. An unnormalized identity path runs from the input to the output. This trains more reliably. I use the pre-LN form.

**Positional representation.** Sinusoidal functions extrapolate to a position that the model never saw. A learned table cannot do this, because position 200 has no vector in a table of 128. A learned table is adequate for my task. My sequences have a fixed maximum length of 128, and I crop the context to the last 128 tokens during generation.

## 3 Implementation Explanation

### 3.1 The format of one training sequence

I build one sequence for each molecule in `mol/data.py`, in `SmilesDataset.__getitem__`.

```
<bos> <qed_k> <logp_k>  C C ( = O ) O c 1 c c c c c 1 ...  <eos> <pad> <pad> ...
|------- prompt -------| |--------- SMILES characters ---|  |---- padding ----|
```

`<qed_k>` and `<logp_k>` are property tokens. QED is a drug-likeness score. LogP is a measure of lipophilicity. RDKit computes both values for each molecule. I put the values into 10 quantile bins. I compute the bin edges from the training split only, so the validation data does not influence them.

The target sequence is the input sequence, moved one position to the left. I set the target to $-100$ on the two property tokens and on the padding. The loss function ignores $-100$. The property tokens are therefore a prompt. They are not a prediction target.

During training I replace each property token with `<qed?>` or `<logp?>`. The probability is `cond_dropout`, and each token is dropped separately. One model can then do two things. It can generate without a condition. It can also accept a target property.

**The tokenizer.** `gpt/tokenizer.py` works at character level. The identifiers are in a fixed order: 4 base special tokens, then 22 property tokens, then the distinct characters of the training split in sorted order. The vocabulary size $V$ therefore comes from the data. On my 16,526-molecule subset, 25 characters occur and $V$ is 51.

I chose character level for a reason. SMILES has no words. A subword vocabulary would hold common fragments, such as a complete benzene ring. The model could then copy a fragment instead of learning the grammar. At character level the model must close every bracket itself.

### 3.2 Notation and model size

| Symbol | Meaning | Value in my model |
|---|---|---|
| $B$ | batch size | 256 |
| $T$ | sequence length | up to 128 |
| $C$ | model width (`n_embd`) | 256 |
| $H$ | number of heads (`n_head`) | 8 |
| $D$ | head dimension ($C/H$) | 32 |
| $V$ | vocabulary size | from the data |
| $N$ | number of blocks (`n_layer`) | 8 |

Table 4: Notation used in this report.

### 3.3 The model architecture

Figure 1 shows the complete data flow. Every edge carries a tensor, and I give its shape.

The meaning of the output is important. `logits[b,t,:]` holds the scores for the token **after** position $t$. One forward pass therefore gives a prediction at every position.

The two embeddings answer two different questions. The token embedding answers the question: what is this character? The position embedding answers the question: where is this character? The model adds the two vectors. The sum is the start of the residual stream. Every block then reads the stream and adds a small change to it.

### 3.4 Inside causal self-attention

Table 5 follows one block, from `gpt/model.py:52-65`.

| Step | Operation | Shape |
|---|---|---|
| Input | `x` | `(B,T,C)` |
| Project | `qkv(x)` | `(B,T,3C)` |
| Split | `q, k, v` | 3× `(B,T,C)` |
| Separate the heads | `view(B,T,H,D).transpose(1,2)` | 3× `(B,H,T,D)` |
| Score | `q @ k.T / sqrt(D)` | `(B,H,T,T)` |
| Mask | `masked_fill(tril == 0, -inf)` | `(B,H,T,T)` |
| Normalize | `softmax(dim=-1)` | `(B,H,T,T)` |
| Collect the values | `weights @ v` | `(B,H,T,D)` |
| Join the heads | `transpose(1,2).view(B,T,C)` | `(B,T,C)` |
| Project | `proj(...)` | `(B,T,C)` |

Table 5: The tensor shapes through one attention sublayer.

Three points connect my code to the paper.

**One matrix for Q, K and V.** I compute the three projections in one `nn.Linear` of width $3C$, and then I split the result. This is equal to three separate matrices. One large matrix multiplication is faster than three small ones.

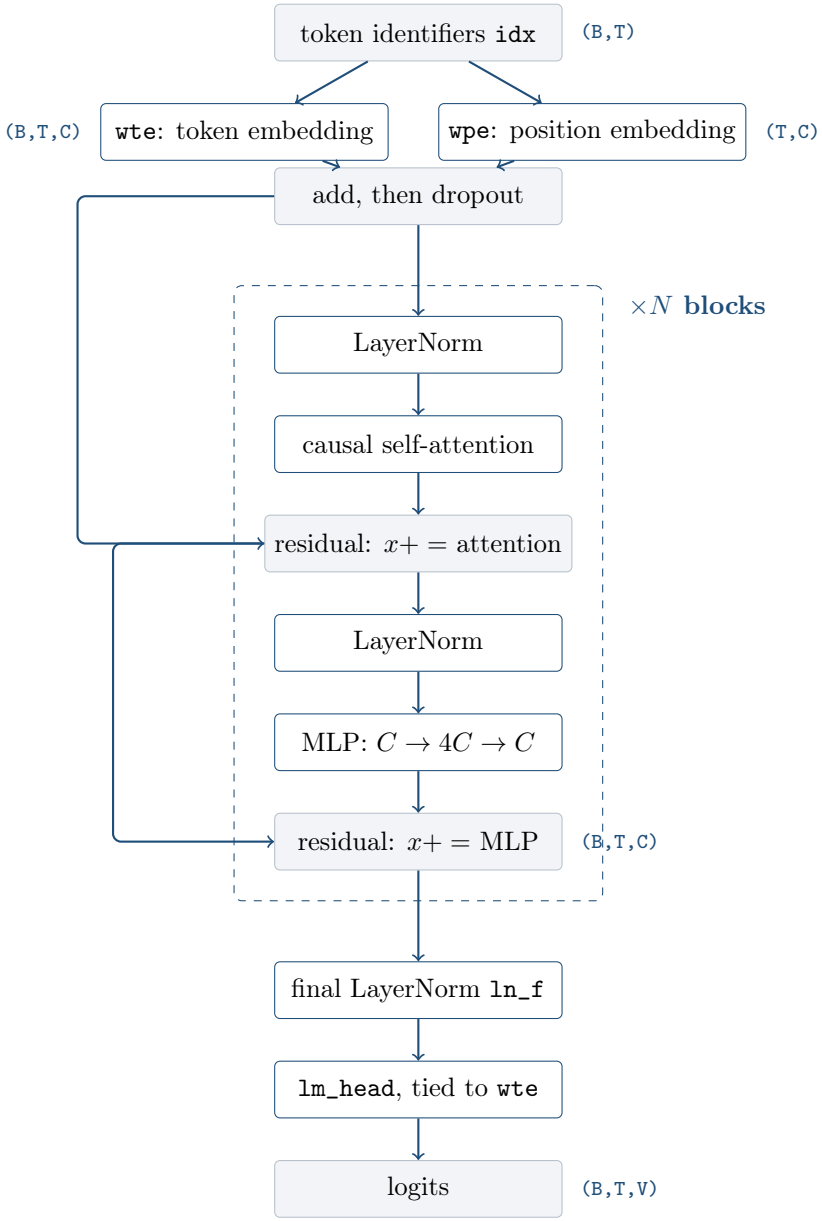



Figure 1: The data flow from token identifiers to next-token logits. The two residual connections give an identity path through every block.

**Multi-head attention is a reshape and not a loop.** I move the head axis next to the batch axis with `view` and `transpose`. All 8 heads then compute in one batched matrix multiplication. This is the concatenation $\text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$ from the paper. The reverse reshape performs the concatenation. My `proj` layer is $W^O$.

**The shape `(B,H,T,T)` is the attention map.** Element `[b,h,i,j]` is the weight that head $h$ gives to position $j$ when it computes position $i$. The mask makes the upper triangle zero. This tensor is also the memory cost of attention. It grows with $T^2$, which is the reason for my context limit of 128.

### 3.5 The loss

I use cross-entropy in `gpt/model.py:150`. I flatten the logits to `(B*T, V)`. I flatten the targets to `(B*T,)`. I set `ignore_index=-100`.

The loss is the mean negative log-likelihood of the correct next character. It covers all positions that are not masked. This is the autoregressive factorization in Equation 3.

$$p(x) = \prod_t p(x_t \mid x_{<t}) \tag{3}$$

The causal mask makes this correct. Each factor uses only the characters before position $t$.

### 3.6 Text generation

I generate one token at a time in `gpt/generate.py` and `mol/sample.py`.

1. Run the model on the current sequence.
2. Take the last position, `logits[:, -1, :]`.
3. Divide the logits by the temperature.
4. Keep the top $k$ logits, if the caller asks for this.
5. Apply softmax and sample. If the temperature is 0, take the `argmax` instead.
6. Append the new token.
7. Stop at `<eos>`, or at `block_size`.

I crop the context to the last `block_size` tokens in step 1. A fixed table of 128 position vectors is therefore sufficient.

`mol/sample.py` does the same work for a complete batch. It records the rows that produced `<eos>`. It writes padding into those rows after that point. This makes the generation of 5000 molecules practical.

## 4 Experiments and Findings

`scripts/experiments.py` produces the results in this section. Every table is also in `results/experiments/summa`

**Status.** The values marked ***TBD*** are not measured yet. The training run and the experiment kernel must finish first. I do not write a finding before I see the measurement.

### 4.1 Training and validation loss

The left plot shows the two losses. The right plot shows the three generation metrics. I measure the metrics on fresh samples at every evaluation step.

The second plot is the more useful one. A falling loss shows that the model predicts characters better. A rising validity shows that the model learned the grammar. The model must close every bracket and pair every ring digit to produce a valid molecule.

**Interpretation.** ***TBD***

| Item | Value |
|---|---|
| Steps | ***TBD*** |
| Final training loss | ***TBD*** |
| Final validation loss | ***TBD*** |
| Parameters | ***TBD*** |
| Run time | ***TBD*** minutes on a Kaggle T4 |

Table 6: The main training run.

*Figure pending.* The plot `../results/smiles_loss.png` appears here after the experiment run finishes.

Figure 2: Left: training and validation loss. Right: validity, uniqueness and novelty, measured on fresh samples at every evaluation step.

### 4.2 The effect of temperature

Temperature divides the logits before the softmax. The effect is a change of shape in the probability distribution.

- A low temperature makes the distribution sharp. The model almost always takes the most probable character.
- A high temperature makes the distribution flat. Rare characters become more probable.
- Temperature 0 is greedy decoding. The model always takes the most probable character.

Greedy decoding has a consequence that I must state. The model gets the same prompt every time. It therefore produces the same string every time. Uniqueness falls to one molecule.
**Interpretation.** ***TBD***

### 4.3 The effect of prompt length and context

I made two measurements.

**Measurement 1: the loss at each position.** I compute the mean cross-entropy at each position index over the validation set. The question is simple. Does more context make the next character easier to predict?

**Measurement 2: completion of a real prefix.** I take a molecule from the validation set. I keep the first $k$ characters. I ask the model to complete the string. I then score the complete string with RDKit. I use $k = 0$, 5, 10, 20 and 40. The case $k = 0$ is the unconditional baseline.

This measurement has two possible outcomes, and both are informative. A long prefix gives the model more context, so the completion can become more reliable. However, a long prefix also makes a commitment. A prefix with two open rings must close both rings. The model can fail if it does not have enough remaining positions.

| Metric | Meaning | Score |
|---|---|---|
| Validity | the generated strings that RDKit can parse | ***TBD*** |
| Uniqueness | the distinct molecules among the valid ones | ***TBD*** |
| Novelty | the valid molecules that are not in the training set | ***TBD*** |

Table 7: Generation metrics from the final checkpoint.

*Figure pending.* The plot `../results/experiments/temperature.png` appears here after the experiment run finishes.

Figure 3: Validity, uniqueness and novelty against the sampling temperature.

| Temperature | Validity | Uniqueness | Novelty | Mean QED | Mean logP |
|---|---|---|---|---|---|
| 0 (greedy) | *TBD* | *TBD* | *TBD* | *TBD* | *TBD* |
| 0.5 | *TBD* | *TBD* | *TBD* | *TBD* | *TBD* |
| 0.7 | *TBD* | *TBD* | *TBD* | *TBD* | *TBD* |
| 0.8 | *TBD* | *TBD* | *TBD* | *TBD* | *TBD* |
| 1.0 | *TBD* | *TBD* | *TBD* | *TBD* | *TBD* |
| 1.2 | *TBD* | *TBD* | *TBD* | *TBD* | *TBD* |
| 1.5 | *TBD* | *TBD* | *TBD* | *TBD* | *TBD* |

Table 8: The effect of temperature on 2000 unconditional samples per setting.

*Figure pending.* The plot `../results/experiments/context.png` appears here after the experiment run finishes.

Figure 4: Left: mean cross-entropy at each position index. Right: validity of completions against the prefix length.

| Prefix length | Validity | Uniqueness | Novelty | Mean length |
|---|---|---|---|---|
| 0 | *TBD* | *TBD* | *TBD* | *TBD* |
| 5 | *TBD* | *TBD* | *TBD* | *TBD* |
| 10 | *TBD* | *TBD* | *TBD* | *TBD* |
| 20 | *TBD* | *TBD* | *TBD* | *TBD* |
| 40 | *TBD* | *TBD* | *TBD* | *TBD* |

Table 9: Completion of real validation prefixes at temperature 1.0.

**Interpretation.** *TBD*

### 4.4 The effect of the architecture

I changed one architectural parameter at a time. I held every other parameter constant. I also held the data subset constant. Any difference in the validation loss therefore comes from the parameter that I changed.

| Axis | Values | What it changes |
|---|---|---|
| `n_layer` | 2, 4, 8 | the depth, which is the number of refinement steps |
| `n_head` | 2, 4, 8 | the number of attention subspaces, at a constant width |
| `n_embd` | 64, 128, 256 | the width of each token representation |

Table 10: The three ablation axes.

**A necessary caveat.** These runs are short. Each run uses 1200 steps on a subset of 200,000 molecules, and a limit of 15 minutes. They are a controlled comparison between settings. They are not converged models. I can therefore make a claim about a relative trend. I cannot make a claim about the best possible quality.

I first tried these runs on a CPU. After 300 steps the validity was still 0.000. A comparison between two such models measures noise and not depth. I moved the runs to a GPU for this reason.

*Figure pending.* The plot `../results/experiments/ablation_n_layer.png` appears here after the experiment run finishes.

Figure 5: Depth: validation loss against `n_layer`.

*Figure pending.* The plot `../results/experiments/ablation_n_head.png` appears here after the experiment run finishes.

Figure 6: Heads: validation loss against `n_head`, at constant `n_embd`.

A note on the `n_head` sweep. I hold `n_embd` at 256. More heads therefore gives a smaller head dimension $D = C/H$. With 2 heads, $D$ is 128. With 8 heads, $D$ is 32. This sweep does not add capacity. It only changes the partition of the same capacity. A non-monotonic result is therefore possible and interesting. It would show a trade-off between the number of subspaces and the size of each subspace.

**Interpretation.** *TBD*

### 4.5 Limitations

**Every output is a candidate and not a drug.** A valid SMILES string that resembles the training distribution proves very little. It does not show that a molecule binds a target. It does not

*Figure pending.* The plot `../results/experiments/ablation_n_embd.png` appears here after the experiment run finishes.

Figure 7: Width: validation loss against `n_embd`.

| Axis | Value | Parameters | Final validation loss | Validity |
|---|---|---|---|---|
| `n_layer` | 2 | *TBD* | *TBD* | *TBD* |
| `n_layer` | 4 | *TBD* | *TBD* | *TBD* |
| `n_layer` | 8 | *TBD* | *TBD* | *TBD* |
| `n_head` | 2 | *TBD* | *TBD* | *TBD* |
| `n_head` | 4 | *TBD* | *TBD* | *TBD* |
| `n_head` | 8 | *TBD* | *TBD* | *TBD* |
| `n_embd` | 64 | *TBD* | *TBD* | *TBD* |
| `n_embd` | 128 | *TBD* | *TBD* | *TBD* |
| `n_embd` | 256 | *TBD* | *TBD* | *TBD* |

Table 11: The effect of three architectural parameters, one at a time.

show that a chemist can synthesize the molecule. It does not show that the molecule is safe. Those questions need docking simulations and laboratory work. My result is narrower. The model learned the SMILES grammar and the shape of drug-like chemical space from next-character prediction alone.

**The model does not guarantee validity.** The grammar is learned and not enforced. I could add a constrained decoder that rejects an illegal character. I did not do this, because validity is my measurement. An enforced grammar would make the measurement meaningless.

**The context is limited to 128 tokens.** Attention costs $T^2$ in memory. I dropped every molecule with more than 100 characters during data preparation.

**The dataset is a subset.** I used 600,000 molecules and not the complete MOSES set. RDKit canonicalization and the QED calculation run on one thread. The complete set would use most of the GPU session for data preparation alone.

### 4.6 Implementation challenges

**The attention mask must be applied before the softmax.** A mask after the softmax leaves rows that do not sum to 1. The output is then not a weighted average, and the loss becomes difficult to interpret.

**A logged number did not match the data in use.** I added a `max_rows` option to make the ablation runs small. The training script then printed the row count from `meta.json`. That file holds the count before the limit. A capped run therefore reported the complete dataset size. The numbers in a report must come from the data in use, so I corrected the script to report the rows that it loads.

**The property tokens must not contribute to the loss.** The property tokens are an input and not a target. I set the target to $-100$ at those two positions. Without this mask the model would learn to predict the property bin of a molecule that it has not yet written.

**A shared value list does not suit three different axes.** My first ablation command passed one list of values for all three axes. The value 64 is valid for `n_embd`, but the value 2 is not, because `n_embd` must divide by `n_head`. The script skipped that axis without an error. I removed the shared list, so each axis now uses its own values.

### 4.7 Lessons learned

**A measurement is better than an opinion.** RDKit parses a string, or it does not. This made the project tractable. I did not need to judge the quality of the output myself.

**The correct experiment needs the correct hardware.** I wasted time on a CPU ablation. The models did not learn enough to be comparable. A short run on a GPU gives a usable comparison. A long run on a CPU does not.

**Shapes are the best documentation.** I wrote the tensor shape in a comment on almost every line of `gpt/model.py`. A shape comment catches an error immediately. The transpose between `(B,T,H,D)` and `(B,H,T,D)` is easy to get wrong, and a wrong order mixes the heads together.

## References


1. Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. `https://arxiv.org/abs/1706.03762`
2. Radford, A., et al. (2019). Language models are unsupervised multitask learners. *OpenAI*.
3. Polykovskiy, D., et al. (2020). Molecular Sets (MOSES): a benchmarking platform for molecular generation models. *Frontiers in Pharmacology*.
4. Karpathy, A. (2022). Let's build GPT: from scratch, in code, spelled out.
5. Alammar, J. (2018). The illustrated Transformer. `https://jalammar.github.io/illustrated-transforme`